# 🚀 CP Bootcamp 2026 — Day 12

# Theme: Linked List + Hashing Foundation

> **Main Goal:**
> আজকে তুমি দুইটা important concept শিখবে:

1. **Linked List** → Data কীভাবে dynamically store করা যায়
2. **Hashing** → Fast lookup কীভাবে করা যায়

এতদিন:

```
Array
↓
Fixed Memory
```

আজ:

```
Dynamic Memory
↓
Connected Nodes
```

এবং:

```
Search O(n)

↓

Hashing

↓

Average O(1)
```

**Estimated Time:** 6–7 hours

# 📖 Day 12 Chapter Map

```
Day 12
│
├── Chapter 1 — Linked List Mental Model
│
├── Chapter 2 — Node & Pointer Concept
│
├── Chapter 3 — Singly Linked List Implementation
│
├── Chapter 4 — Linked List Operations
│
├── Chapter 5 — Linked List Problem Patterns
│
├── Chapter 6 — Hashing Mental Model
│
├── Chapter 7 — Hash Table & Hash Function
│
├── Chapter 8 — Frequency Counting Pattern
│
├── Chapter 9 — Hashing Applications in CP
│
└── Chapter 10 — Day 12 Assignment & Practice
```

# Chapter 1 — Linked List Mental Model

## Array Problem

Array:

```
int arr[100];
```

Memory:

```
id="y4gq2e"
[10][20][30][40]
```

সব element পাশাপাশি থাকে।

Problem:

যদি মাঝখানে নতুন element insert করতে হয়:

```
10 20 30 40
```

Insert:

```
25
```

Need:

```
10 20 25 30 40
```

অনেক element shift করতে হবে।

Linked List এই সমস্যা solve করে।

# Linked List কী?

Linked List হলো:

> অনেকগুলো node যেখানে প্রতিটি node পরের node-এর address রাখে।

Structure:

```
id="m1m8x7"
[10 | next]
       |
       ↓
[20 | next]
       |
       ↓
[30 | next]
       |
       ↓
NULL
```

একটা node:

```
id="8qv6k1"
Data

+

Next Address
```

# Array vs Linked List

| Array | Linked List |
| --- | --- |
| Continuous memory | Random memory |
| Fixed size | Dynamic size |
| Fast indexing | Slow indexing |

| Array | Linked List |
| --- | --- |
| Insert costly | Insert easier |
| arr[i] possible | No direct index |

# Chapter 2 — Node & Pointer Concept

Linked List বুঝতে Pointer দরকার।

## Pointer কী?

Pointer হলো:

> একটি variable যেটা অন্য variable-এর address রাখে।

Example:

```
int x = 10;

int *ptr = &x;
```

Meaning:

```
ptr
 ↓
x
```

# Node Structure in C

Linked List-এর node:

```
struct Node
{
    int data;

    struct Node *next;
};
```

Meaning:

একটা node-এর:

```
data

+
next pointer
```

আছে।

Example:

```
id="l6m9nw"
Node

data = 10

next = address of next node
```

# Creating Node

```
struct Node *newNode;

newNode = malloc(sizeof(struct Node));
```

Memory:

```
Heap
 ↓

[ data | next ]
```

# Chapter 3 — Singly Linked List Implementation

Singly Linked List:

একদিকে link:

```
id="f2yq0w"
10 → 20 → 30 → NULL
```

# Create Node Function

```
struct Node* createNode(int value)
{
    struct Node* newNode;

    newNode = malloc(sizeof(struct Node));


    newNode→data = value;

    newNode→next = NULL;


    return newNode;
}
```

# Linking Nodes

Example:

```
node1→next = node2;
```

Result:

```
id="0q7x2h"
node1

  ↓

node2
```

# Full Structure

```
head

↓

[10|next]

↓

[20|next]

↓

[30|NULL]
```

# Head কী?

Head হলো:

> প্রথম node-এর address।

Example:

```
struct Node *head;
```

Empty list:

```
head = NULL;
```

# Chapter 4 — Linked List Operations

## 1. Traversal

Array:

```
for(i=0;i<n;i++)
```

Linked List:

Pointer দিয়ে move করতে হয়।

Example:

```
struct Node *temp=head;


while(temp≠NULL)
{
    printf("%d ",temp→data);

    temp=temp→next;
}
```

Flow:

```
head
↓
10
↓
20
↓
30
↓
NULL
```

# 2. Insert at Beginning

Before:

```
10 → 20 → 30
```

Insert:

```
5
```

Step:

New node:

```
5
```

Connect:

```
5 → 10 → 20 → 30
```

Code:

```
newNode→next=head;

head=newNode;
```

# 3. Insert at End

Need:

শেষ পর্যন্ত যেতে হবে।

```
temp=head;


while(temp→next≠NULL)
{
    temp=temp→next;
}
```

Then:

```
temp→next=newNode;
```

# 4. Delete Beginning

Before:

```
10 → 20 → 30
```

Remove:

```
10
```

Update:

```
head=head→next;
```

Result:

```
20 → 30
```

# 5. Search

Linear search-এর মতো।

```
while(temp≠NULL)
{
    if(temp→data==target)
    {
        return 1;
    }

    temp=temp→next;
}
```

Complexity:

```
O(n)
```

# Chapter 5 — Linked List Problem Patterns

# Pattern 1 — Reverse Linked List

Original:

```
1 → 2 → 3 → NULL
```

Reverse:

```
3 → 2 → 1 → NULL
```

Need three pointers:

```
prev
```

```
current
```

```
next
```

Mental Model:

```
Save next

↓

Reverse link

↓

Move forward
```

# Pattern 2 — Fast & Slow Pointer

এইটা Day 9-এর Two Pointer-এর advanced version।

Use:

```
slow

fast
```

Applications:

## Find Middle Node

Example:

```
1 → 2 → 3 → 4 → 5
```

Slow:

```
1 → 2 → 3
```

Fast:

```
1 → 3 → 5
```

Slow middle-এ থাকে।

## Detect Cycle

যদি:

```
1 → 2 → 3 → 4
      ↑       ↓
      ← ← ←
```

cycle আছে।

Fast এবং slow এক জায়গায় মিলবে।

# Chapter 6 — Hashing Mental Model

এখন Linked List থেকে অন্য দিকে যাই।

Problem:

Find করা:

```
100000 element
```

Linear search:

```
O(n)
```

Need faster।

Hashing:

> Data-কে এমনভাবে store করা যাতে direct access পাওয়া যায়।

Example:

Student ID:

```
101 → Rafi

102 → Karim

103 → Nabil
```

ID দিয়ে instantly খুঁজে পাওয়া।

# Chapter 7 — Hash Table & Hash Function

Hash Table:

```
id="0v0d0c"
Index

0

1

2

3

4

5

6

7
```

Hash Function:

Value কে index বানায়।

Example:

```
key = 25
```

Formula:

```
key % size
```

Size:

```
10
```

Result:

```
25 % 10 = 5
```

Store:

```
index 5
```

# Collision

দুইটা key একই index দিলে:

Example:

```
15 % 10 = 5
```

```
25 % 10 = 5
```

দুইটাই index 5।

এটাকে বলে:

```
Collision
```

Solution:

# Chaining

এক index-এ linked list:

```
5

↓

15 → 25 → 35
```

# Chapter 8 — Frequency Counting Pattern

CP-তে Hashing-এর সবচেয়ে common use।

Problem:

Count frequency।

Example:

Array:

```
1 2 2 3 3 3
```

Need:

```
1 → 1

2 → 2

3 → 3
```

Without Hash:

Nested loop:

```
O(n²)
```

Hash:

```
frequency[value]++;
```

C Array Hash:

যদি value ছোট:

```
int freq[1000]={0};
```

Example:

```
for(int i=0;i<n;i++)
{
    freq[arr[i]]++;
}
```

# Chapter 9 — Hashing Applications in CP

# 1. Duplicate Detection

Problem:

Array:

```
1 2 3 2
```

Duplicate?

Hash:

```
seen[value]
```

# 2. Frequency Count

Most common।

# 3. Two Sum

Day 9:

Sorted + Two Pointer।

Alternative:

Hash:

```
Need = target-current
```

Example:

Array:

```
2 7 11 15
```

Target:

```
9
```

2 দেখলাম।

Need:

```
7
```

Hash check।

Found।

# 4. Character Frequency

String:

```
hello
```

Count:

```
h=1

e=1

l=2

o=1
```

# Chapter 10 — Day 12 Assignment

## Part A — Linked List

Implement:

1. Create Node
2. Insert Beginning
3. Insert End
4. Delete Beginning
5. Search Element
6. Traverse List

# Part B — Linked List Problems

Solve:

1. Reverse Linked List
2. Find Middle Node
3. Detect Cycle
4. Count Nodes

# Part C — Hashing Practice

Implement:

1. Frequency Counter
2. Duplicate Finder
3. Character Frequency
4. Two Sum using Hash

# Part D — Dry Run

Linked List:

```
10 → 20 → 30 → NULL
```

Insert 5 at beginning

Show:

```
Old Head:

New Node:

New Head:
```

# Part E — Pattern Recognition

Problem:

> Count how many times each number appears.

Pattern:

```
?
```

Problem:

> Find middle of linked list.

Pattern:

```
?
```

Problem:

> Undo operation.

Pattern:

?

# 📚 Glossary Update

Add:

Linked List

Node

Pointer

Address

Head

Next Pointer

Dynamic Memory

Singly Linked List

Traversal

Fast Pointer

Slow Pointer

Cycle

Hashing

Hash Table

Hash Function

Collision

Chaining

Frequency Array

Key Value Pair

# 📚 Pattern Library Update

Add:

```
## Fast Slow Pointer Pattern

Recognition:

- Middle element
- Cycle detection


Mental Model:

slow → one step

fast → two steps


Applications:

- Linked List middle
- Cycle detection
```

## Hash Frequency Pattern

Recognition:

- Count occurrence
- Duplicate detection
- Lookup required

Mental Model:

Element

↓

Hash Index

↓

Store Information

Complexity:

Average O(1)

# ✅ Day 12 Completion Checklist

[ ] Linked List concept বুঝি

[ ] Node structure বুঝি

[ ] Pointer বুঝি

[ ] Head বুঝি

[ ] Traversal করতে পারি

[ ] Insert/Delete করতে পারি

[ ] Reverse linked list idea বুঝি

[ ] Fast Slow Pointer বুঝি

[ ] Hashing concept বুঝি

[ ] Hash Function বুঝি

[ ] Collision বুঝি

[ ] Frequency counting করতে পারি

[ ] Hash দিয়ে duplicate detect করতে পারি

# 🏁 Day 12 Final Mental Model

```
Problem

↓

Need storage?

↓

Need order?
→ Linked List

Need fast lookup?
→ Hashing

Need frequency?
→ Hash Table

Need middle/cycle?
→ Fast Slow Pointer
```

Day 12 শেষে:

Day 1-5
Pattern Foundation

+

Day 6
Functions

+

Day 7
Searching

+

Day 8
Sorting

+

Day 9
Optimization

+

Day 10
Recursion

+

Day 11
Stack Queue

+

Day 12
Linked List + Hashing

পরের:

# Day 13 — Trees + Binary Search Tree Foundation 🌳